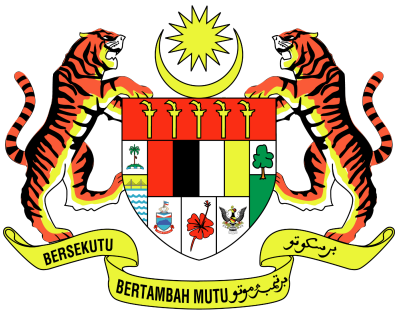
CT125 DIGITAL TECHNOLOGY



KEMENTERIAN PENDIDIKAN
BAHAGIAN MATRIKULASI

FINAL PROJECT REPORT & PRESENTATION

SEMESTER II (2025/2026)

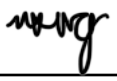
ID PROJECT

P72

No.	Matric Number	ID Number
1.	MC2516116804	P7201
2.	MC2516203257	P7202
3.	MC2516116835	P7203
4.	MC2516116793	P7204
5.	MC2516203243	P7205

CANDIDATE'S DECLARATION

We declare that this **FINAL PROJECT REPORT & PRESENTATION** is our own work except for the citations and summaries of which we acknowledged the source.

No.	Matric Number	ID Number	Signature
1	MC2516116804	P7201	
2	MC2516203257	P7202	
3	MC2516116835	P7203	
4	MC2516116793	P7204	
5	MC2516203243	P7205	

Date: 22 April 2026

TABLE OF CONTENT

1. Introduction **3**

1.1. Problem Statement 3

1.2. Importance of the Problem 3

1.3. Proposed Solution 4

2. Objectives **4**

3. Scope of the Project **5**

3.1. Scope Of Implementation 5

3.2. Scope Of Capabilities 5

3.3. Limitation of Project 6

4. Methodology **7**

4.1. Circuit Diagram 8

4.2. Flowchart 9

5. Specifications **10**

5.1. Hardware 10

5.2. Software 12

6. Implementation **14**

6.1. Implementation of The Design Solution 14

6.2. Device Casing Design and Fabrication 23

6.3. Testing 28

6.4 Result and Discussion 23

7. Conclusion **36**

7.1. Future Work 37

8. References **39**

9. Presentation Video **40**

9.1. Timeframe Table 40

1. Introduction

The general introduction of the project is about one paragraph.

1.1. Problem Statement

In educational environments such as matriculation colleges and boarding schools, student safety is an increasing concern as many students live away from their families and may face risks such as bullying, neglect, and sexual harassment in areas with limited supervision, including hostels, classrooms, and secluded spaces. Many incidents remain unreported because victims may fear retaliation, feel ashamed, or not know whom to approach, while delayed access to assistance and hidden incident locations often result in late intervention and long-term physical or psychological effects. Existing safety measures such as manual reporting systems, CCTV monitoring, and warden patrols are often insufficient for providing immediate alerts or real-time response during emergencies. Therefore, there is an urgent need for a reliable, discreet, and efficient safety system that allows students to report emergencies instantly while enabling authorities to respond quickly, with a strong focus on privacy, accessibility, and ease of use.

1.2. Importance of the Problem

Ensuring student safety is essential for maintaining a secure and supportive educational environment, especially in matriculation colleges and boarding schools where students live under institutional care. Unreported cases of bullying, neglect, or harassment can cause serious emotional distress, anxiety, and long-term psychological effects, while the absence of an effective and immediate reporting mechanism reduces students' confidence in the institution and limits authorities' ability to respond promptly. This not only places students at continued risk but also affects the institution's trust and reputation. Therefore, addressing this issue is crucial to creating a safer and more responsive learning environment by establishing an efficient safety system and clear communication channels that provide timely assistance, prevent further incidents, and promote a culture of care, trust, and accountability within the campus community.

1.3. Proposed Solution

IoSafe (Sos Device)

The proposed solution, IoSafe (SOS Device) is an innovative and practical personal safety device designed to enhance user security through real-time location tracking and rapid emergency alert activation. The device functions by obtaining the user's live GPS coordinates and triggering an immediate distress signal when activated, allowing emergency contacts or responsible parties to respond more quickly during critical situations. Unlike smartphone-dependent solutions, IoSafe provides a dedicated and easily accessible activation mechanism, enabling users to send alerts instantly during panic or high-stress moments. To further improve safety, the device also includes an audible alarm to attract nearby attention and deter potential threats, as well as a simple indicator to confirm that the emergency signal has been successfully activated. Powered by a rechargeable battery and designed for portability and continuous daily use, IoSafe is especially suitable for students, women, the elderly, and travelers who may be more vulnerable in emergency situations. By combining location tracking, fast alert functionality, and user-friendly operation, IoSafe offers a realistic, reliable, and impactful solution for improving personal safety in various real-world environments.

2. Objectives

This project focuses on three key objectives to ensure safety, efficiency, and reliable emergency response using an IoT-based SOS device.

Objective 1 (Identify):

To identify and understand the main problems faced during emergency situations, such as delays in requesting help or difficulties in sharing location, so that an effective IoT-based SOS solution can be developed.

Objective 2 (Develop):

To design and develop an IoT-based SOS device that can quickly send emergency alerts and location information to selected contacts or authorities, making the process of requesting help faster and easier.

Objective 3 (Evaluate):

To evaluate the effectiveness of the SOS device by collecting feedback from users and testing its performance, ensuring that the system is reliable, responsive, and suitable for long-term use.

3. Scope of the Project

3.1. Scope Of Implementation

This project will be implemented within a college environment, where a student can send an emergency signal directly to the warden's monitoring system by pressing the SOS button on the device. The IoSafe system integrates multiple IoT components, including a GPS module and accelerometer sensor, all managed by the ESP32 microcontroller.

When the panic button is activated or when the accelerometer detects abnormal movement or impact beyond a predefined threshold, the system immediately transmits a real-time alert containing the user's location and device ID to selected emergency contacts (e.g., parents, guardians, or wardens). These alerts and location data are displayed on a web-based dashboard, enabling real-time monitoring and quick decision-making during emergencies. This implementation ensures efficient communication between the user and the monitoring authority within the institution.

3.2. Scope Of Capabilities

The IoSafe device is designed to enhance personal safety by enabling users to signal distress in emergency situations such as harassment, accidents, fainting, or being followed. Once activated, the system:

- Sends real-time GPS location data to the designated recipients (warden and parent).
- Triggers audible and visual indicators to confirm successful transmission.
- Allows remote monitoring through an integrated IoT dashboard.

  This system empowers users to request immediate assistance, ensuring faster responses and improved situational awareness for the monitoring party.

3.3. Limitation of Project

- **Network Dependence** – The system requires GPS connectivity to transmit data effectively. In areas with weak or no signal coverage, the system's performance may be compromised.
- **False Alarms** – Accidental button activation may trigger unnecessary alerts if no confirmation or cancellation mechanism is integrated.
- **Response Time** – The speed of emergency response depends on how promptly the recipient (parent or warden) reacts to the alert.
- **Power Management** – The device relies on a rechargeable battery, which requires periodic charging and maintenance to ensure readiness during emergencies.
- **Data Accuracy** – GPS precision can be affected by environmental factors such as buildings, weather, or signal interference.
- **User Awareness** – Proper training and awareness are required to ensure users understand how and when to use the device appropriately.
- **Connectivity Integration** – The system's reliability depends on effective synchronization between the device, IoT modules, and the web dashboard.

4. Methodology

4.1 Circuit Diagram

Figure 1 shows the working principle of the IoSafe. The ESP32 microcontroller acts as the main controller, interfacing with several modules including the GPS module, touch sensor, push button, buzzer, gyro accelerometer sensor, and power management circuit. When powered on, the ESP32 initializes all connected components and begins monitoring the touch sensor and gyro accelerometer sensor.

In normal conditions, the system remains in standby mode to conserve power. When the user touches the touch sensor for 3 seconds, the ESP32 activates the GPS module to obtain the current latitude and longitude coordinates. These coordinates are then processed and combined with a timestamp and device ID to form a location data packet. The data is then processed by the ESP32 for emergency location tracking purposes.

After successful activation, the buzzer briefly sounds to indicate that the system has been triggered. The battery management module ensures stable power delivery and recharging through the module charger and the buck converter. Finally, the system re-enters standby mode, awaiting the next touch input, button touched, or motion detection to send a new location update.

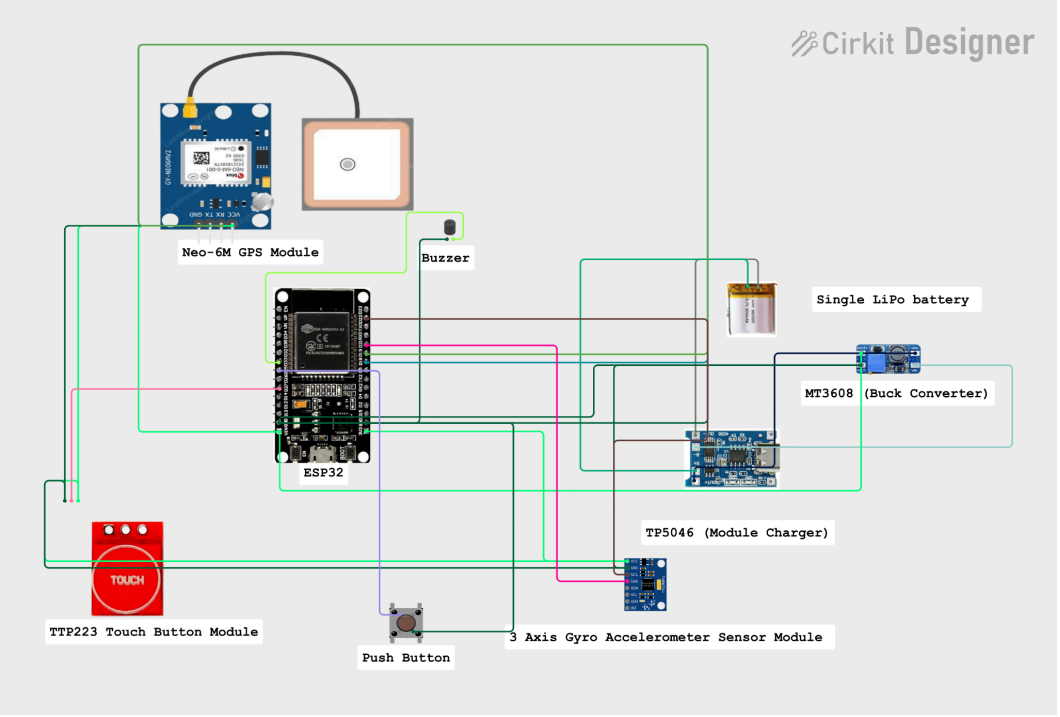


Figure 1: Circuit Diagram of IoSafe Unit

4.2 Flowchart

Figure 2 shows how the IoSafe operates. The system starts by initializing and turning on components such as the ESP32, GPS module, touch sensor, push button, buzzer, and accelerometer sensor. It then waits for an emergency trigger from the SOS button (TTP223 touch sensor). If the button is not pressed, the device stays in standby mode while continuously monitoring for input. When the button is pressed, the system activates the buzzer as an alarm, then reads the GPS coordinates and prepares the emergency alert data containing the GPS location and time. The system then sends the emergency alert data for tracking purposes. Finally, the system returns to standby mode and waits for the next SOS signal.

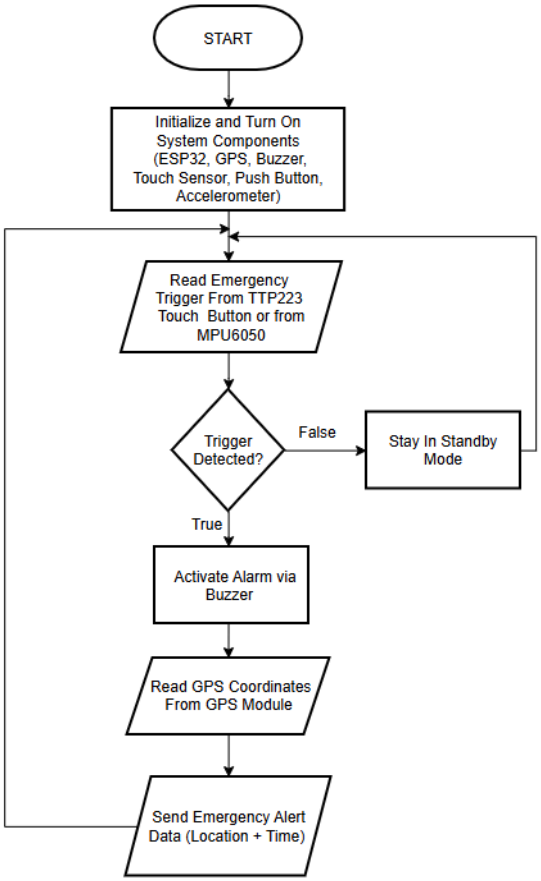
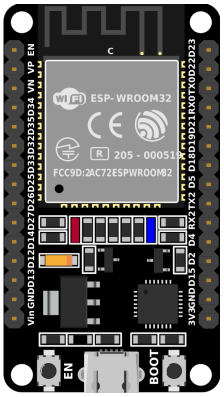
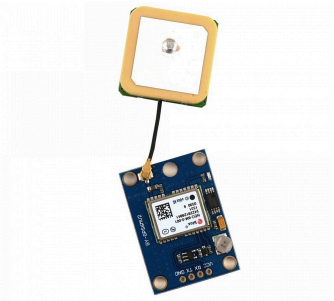
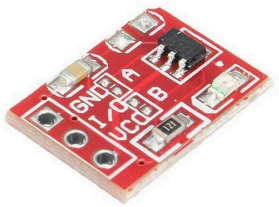


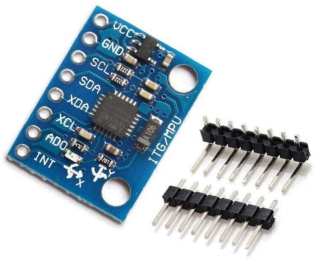
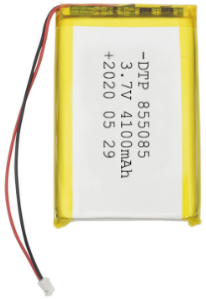
Figure 2: Flowchart of IoSafe Unit

5. Specifications

List of Components & Specifications, Functions of Components and Pictures of Components

a. Hardware

	**ESP32** The ESP32 is used because it has built-in WiFi, faster processing, and low power usage. This makes it suitable for real-time IoT systems and battery-powered devices without needing extra modules.
	**Neo-6M GPS Module** The Neo-6M GPS module was selected for its good accuracy (±5–10 meters) and reliable outdoor performance. It is also cost-effective and widely used, making it suitable for IoT projects.
	**TP223 Touch Button Module** The TTP223 touch sensor was chosen for its fast response and durability, enabling easy SOS activation without mechanical wear.

	**3 Axis Gyro Accelerometer Sensor Module (MPU 6050)** The MPU6050 was selected for its combined accelerometer and gyroscope, enabling detection of abnormal movements such as falls or impacts. This supports automatic SOS triggering and improves user safety.
	**Buzzer** The buzzer is used to confirm SOS activation and alert nearby people in emergency situations.
	**Single LiPo** The LiPo battery was selected for its compact size, high energy density, and rechargeable efficiency, making it ideal for portable, long-term use without constant external power.
	**Module Charger** The TP4056 module was chosen for safe, efficient LiPo charging with built-in overcharge and over-discharge protection, ensuring battery safety and longer lifespan.

	**Buck Converter** The MT3608 buck converter was used to provide stable voltage to all components, ensuring each module receives the correct power level. It helps prevent damage, improves efficiency, and supports stable system operation.

b. Software

	**Arduino IDE** ● Used to write, compile, and upload the ESP32 program using C/C++ language. **Justification:** Arduino IDE was chosen because it is **easy to use and supports many libraries**, allowing efficient development and debugging of the ESP32 system.
	**Firebase** ● It stores and synchronizes data between ESP32 and the dashboard of the webpage. **Justification:** Firebase was selected for its **real-time data synchronization**, enabling instant transmission of emergency alerts without needing a complex backend system.

	**Node.js** ● Allows developer to execute JavaScript code outside of a web browser **Justification:** Node.js is used because of its **fast and event-driven nature**, making it suitable for handling real-time data updates from the IoT system.
	**Telegram Bot API** ● Used to send real-time push notifications to users when an SOS alert is triggered **Justification**: Telegram Bot API was chosen due to its **simplicity**, **fast message delivery**, and ability to send automated notifications without requiring a **dedicated mobile application.** It enhances the system by providing instant alerts directly to users' smartphones.
	**Snapmaker Luban** ● Used to prepare and slice 3D models, generate laser/CNC toolpaths, and control the Snapmaker machine. **Justification**: Luban integrates all fabrication workflows (3D printing, laser, CNC) with straightforward settings and previews, simplifying the production of parts that integrate with your ESP32 hardware.

6. Implementation

This section explains how the proposed IoT SOS device system was built and applied based on the design developed earlier

6.1 Implementation of The Design Solution

The implementation of the IoT-based SOS device was carried out based on the system design and flowchart developed in the previous section. The system integrates hardware components (ESP32, GPS module, touch sensor, buzzer, and accelerometer) with software technologies including Arduino IDE, Firebase, and a web-based monitoring dashboard.

6.1.1 System Architecture

The overall system architecture consists of three main layers:

1. Device Layer (ESP32 System)

The ESP32 acts as the main controller of the IoSafe system, functioning as the core processing unit. It continuously monitors input from the touch sensor (SOS trigger) and the MPU6050 accelerometer sensor (abnormal movement detection). By processing data at the device level, the system ensures immediate response to emergency events without relying on external computation.

The integration of multiple sensors enhances system reliability. The touch sensor enables manual activation during panic situations, while the accelerometer allows automatic detection of sudden impacts or falls. Once triggered, the ESP32 retrieves real-time location data from the Neo-6M GPS module. Although the GPS module provides good accuracy in outdoor environments, its performance may be affected indoors due to limited satellite visibility.

2. Cloud Layer (Firebase Realtime Database)

Firebase Realtime Database is used as the central cloud platform to store and synchronize SOS alert data. The ESP32 transmits data such as device ID, GPS coordinates, and timestamp via WiFi to Firebase in real time.

Firebase is selected due to its low latency and real-time synchronization capability, allowing instant data updates between the device and the monitoring system. The database is structured

in a hierarchical JSON format to support efficient data organization and scalability. For example:

devices/device_id

- status: "SOS"
- latitude: value
- longitude: value
- timestamp: value

This structure enables the system to support multiple devices and ensures efficient data retrieval for monitoring purposes.

3. Application and Notification Layers (Web Dashboard & Telegram Bot)

The application layer consists of a web-based dashboard developed using Node.js, HTML, and JavaScript. The dashboard continuously retrieves real-time data from Firebase and displays important information such as device ID, alert status, and user location. Integration with mapping services (e.g., Google Maps API) allows visualization of GPS coordinates for easier tracking and decision-making.

In addition to the dashboard, a Telegram Bot is integrated as an active notification system. While the dashboard requires manual monitoring, the Telegram Bot automatically sends push notifications whenever an SOS alert is triggered. This ensures that emergency alerts are received instantly, even when users are not actively viewing the dashboard.

The notification includes key information such as alert type, device ID, time, battery status, and a direct link to the dashboard. This significantly improves response time and enhances the overall effectiveness of the IoSafe system..

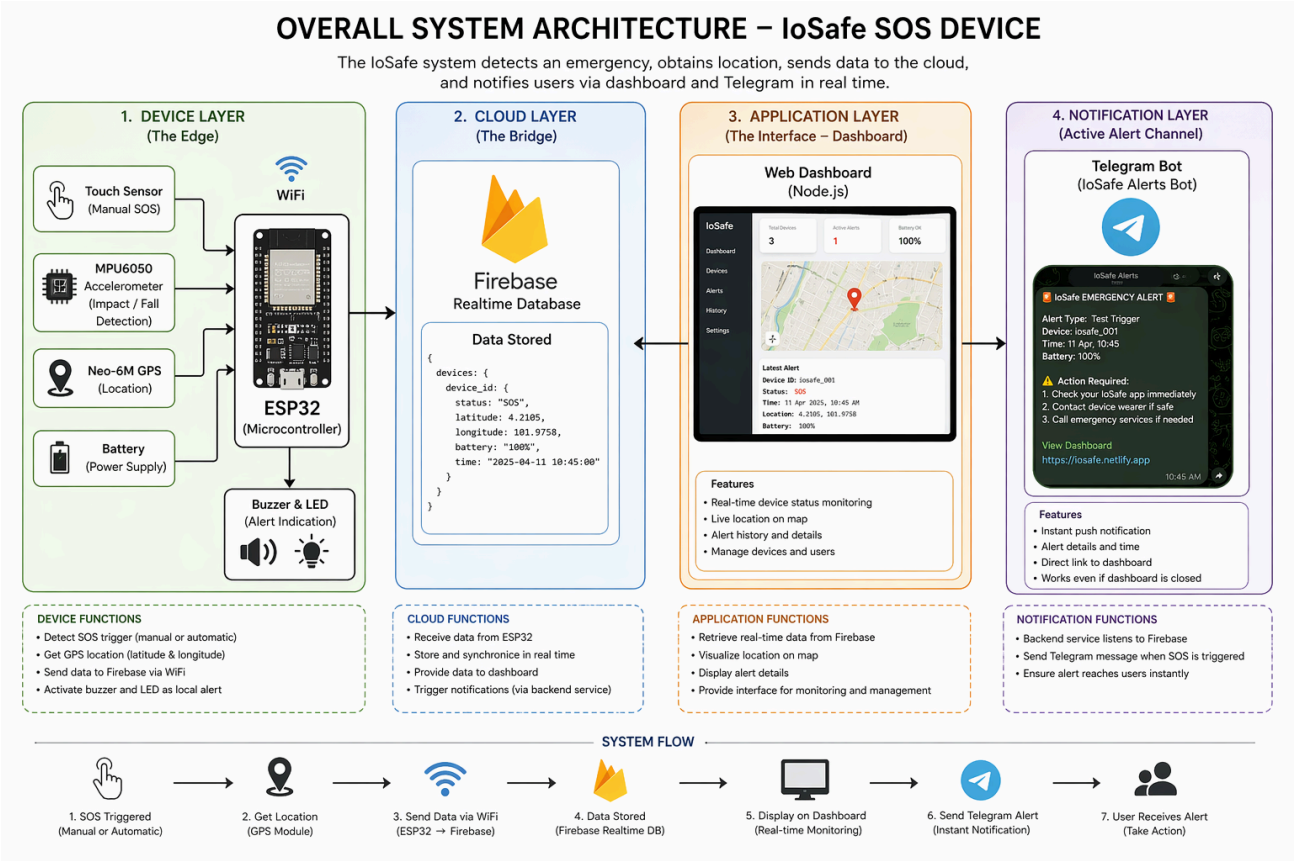


Figure 6.1: Overall System Architecture of IoSafe

Figure 6.1 shows the complete data flow of the system. The ESP32 sends emergency data via WiFi to Firebase, which is then accessed by the web dashboard for real-time monitoring. This architecture ensures fast and reliable communication between the user and the monitoring authority.

6.1.2 ESP32 Program Implementation

The ESP32 was programmed using Arduino IDE in C/C++. The program initializes all components and continuously monitors the SOS trigger.

```
252     case STATE_IMPACT:
253       if (totalAccel > 8.0 && totalAccel < 12.0) {
254         float movement = abs(totalAccel - prevTotalAccel);
255         if (movement < ACTIVITY_THRESHOLD) {
256           Serial.println("\n🚨 FALL DETECTED BY MPU6050!");
257           Serial.printf("   Impact Force: %.2f m/s² (%.2f g)\n", totalAccel, totalAccel/9.81);
258           Serial.printf("   Temperature: %.1f°C\n", temperature);
259           Serial.printf("   Time: %s\n", getCurrentTimeString().c_str());
260
261           activateSOS("mpu_fall", 0);
262
263           fallState = STATE_POST_FALL;
264           fallDetectionStart = now;
265         }
266       }
```

```
664     if (touchHeldDuration >= (unsigned long)settings.sosDuration) {
665       Serial.printf("  ✓ Full %ds - SOS!\n", targetSeconds);
666       activateSOS("touch_release_full", touchHeldDuration);
667     } else if (touchHeldDuration >= 1000) {
668       Serial.printf("  ⚠ %ds/%ds - Partial SOS\n", secondsHeld, targetSeconds);
669       activateSOS("touch_release_partial", touchHeldDuration);
670     } else {
671       Serial.println("  ✗ Cancelled");
672       tickBuzzer();
673       historySavedForCurrentSOS = false;
674     }
```

Figure 6.2: ESP32 Arduino Code for SOS Trigger Detection

Figure 6.2 shows the condition used to detect when the touch sensor is activated. When the sensor value drops below a threshold, the system triggers the SOS function.

After retrieving GPS data, the system formats the data and sends it to Firebase:

```
// ============== DEVICE SETTINGS ==============
void checkDeviceSettings() {
  if (!Firebase.ready()) return;
  bool success = false;
  int retries = 0;
  while (!success && retries < 3) {
    if (Firebase.RTDB.getJSON(&fbdo, "/devices/" + deviceID + "/settings")) {
      FirebaseJson json;
      json.setJsonData(fbdo.jsonString());
      FirebaseJsonData result;

      json.get(result, "sosDuration");
      if (result.success) {
        int newDuration = result.intValue;
        if (newDuration >= 1000 && newDuration <= 30000 && newDuration != settings.sosDuration) {
          settings.sosDuration = newDuration;
          Serial.printf("✓ SOS duration updated: %d ms\n", settings.sosDuration);
        }
      }

      json.get(result, "buzzerEnabled");
      if (result.success) {
        bool newBuzzer = result.boolValue;
        if (newBuzzer != settings.buzzerEnabled) {
          settings.buzzerEnabled = newBuzzer;
          Serial.printf("✓ Buzzer setting updated: %s\n", settings.buzzerEnabled ? "ON" : "OFF");
        }
      }

      json.get(result, "mpuEnabled");
      if (result.success) {
        bool newMpu = result.boolValue;
        if (newMpu != settings.mpuEnabled) {
          settings.mpuEnabled = newMpu;
          Serial.printf("✓ MPU setting updated: %s\n", settings.mpuEnabled ? "ON" : "OFF");
        }
      }

      json.get(result, "fallThreshold");
      if (result.success) {
        float newThreshold = result.floatValue;
        if (newThreshold >= 10.0 && newThreshold <= 50.0 && newThreshold != settings.fallThreshold) {
          settings.fallThreshold = newThreshold;
          Serial.printf("✓ Fall threshold updated: %.1f m/s²\n", settings.fallThreshold);
        }
      }

      success = true;
    } else {
      retries++;
      Serial.printf("X Settings read failed (attempt %d): %s\n", retries, fbdo.errorReason().c_str());
      if (retries < 3) delay(500);
    }
  }
}
```

Figure 6.3: ESP32 Sending Data to Firebase

Figure 6.3 shows how the ESP32 sends SOS data such as location and device ID to Firebase using WiFi. This enables real-time data storage and access.

6.1.3 Firebase Database Structure

Firebase Realtime Database is used to store SOS alert data in a structured format.

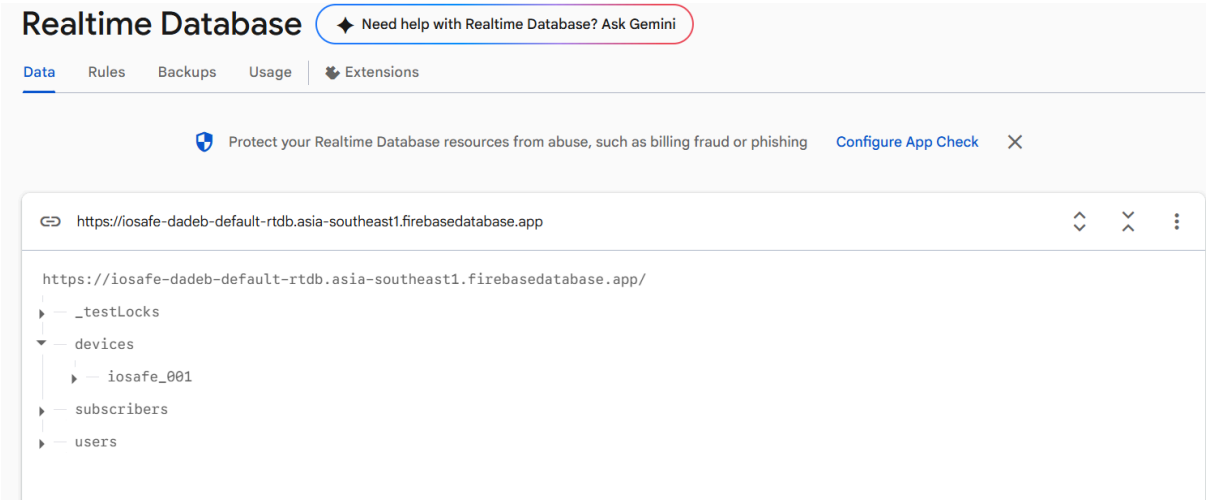


Figure 6.4: Firebase Realtime Database Structure

Figure 6.4 shows how data is organized in Firebase. Each SOS alert contains device ID, location coordinates, and timestamp. This structure allows efficient data retrieval and real-time updates.

6.1.4 Web Dashboard Implementation

The web dashboard was developed using HTML, JavaScript, and Node.js to display real-time alerts.

```
534     try {
535       const deviceRef = ref(db, `devices/${userProfile.deviceId}`);
536
537       if (status) {
538         // Triggering SOS - generate unique alertId using crypto if available
539         const alertId = typeof crypto !== 'undefined' && crypto.randomUUID
540           ? crypto.randomUUID()
541           : `${Date.now()}_${Math.random().toString(36).substr(2, 9)}`;
542
543         const updateData = {
544           sos: true,
545           sensor: "web_test",
546           alertNotified: false,
547           alertId: alertId,
548           sosTriggeredAt: Date.now(),
549           sosTriggeredAtString: new Date().toISOString(),
550           lastUpdate: Date.now(),
551           lastUpdateString: new Date().toISOString(),
552         };
553
554         console.log("Triggering SOS with ID:", alertId);
555         await update(deviceRef, updateData);
556
557         // ✅ FIXED: Reset refs BEFORE Firebase update to ensure listener catches it
558         // Actually, reset refs to false so the listener sees a transition
559         lastSosStateRef.current = false; // This allows the listener to detect the new SOS
560
561       } else {
562         // Clearing SOS
563         const updateData = {
564           sos: false,
565           sensor: "none",
566           alertNotified: false,
567           alertId: null, // ✅ FIXED: Use null instead of 0 for clarity
568           sosClearedAt: Date.now(),
569           lastUpdate: Date.now(),
570         };
571
572         console.log("Clearing SOS");
573         await update(deviceRef, updateData);
574
575         // Reset refs
576         lastSosStateRef.current = false;
577         lastAlertIdRef.current = null;
578       }
579
580       console.log("✓ Firebase update successful");
581     } catch (error) {
582       console.error("✗ Firebase update failed:", error);
583       alert("Failed: " + error.message);
584     } finally {
585       setUpdating(false);
586     }
587   }, [userProfile?.deviceId]);
```

Figure 6.5: JavaScript Code for Retrieving Firebase Data

Figure 6.5 shows the JavaScript function used to retrieve data from Firebase. The system continuously listens for updates and displays new alerts instantly.

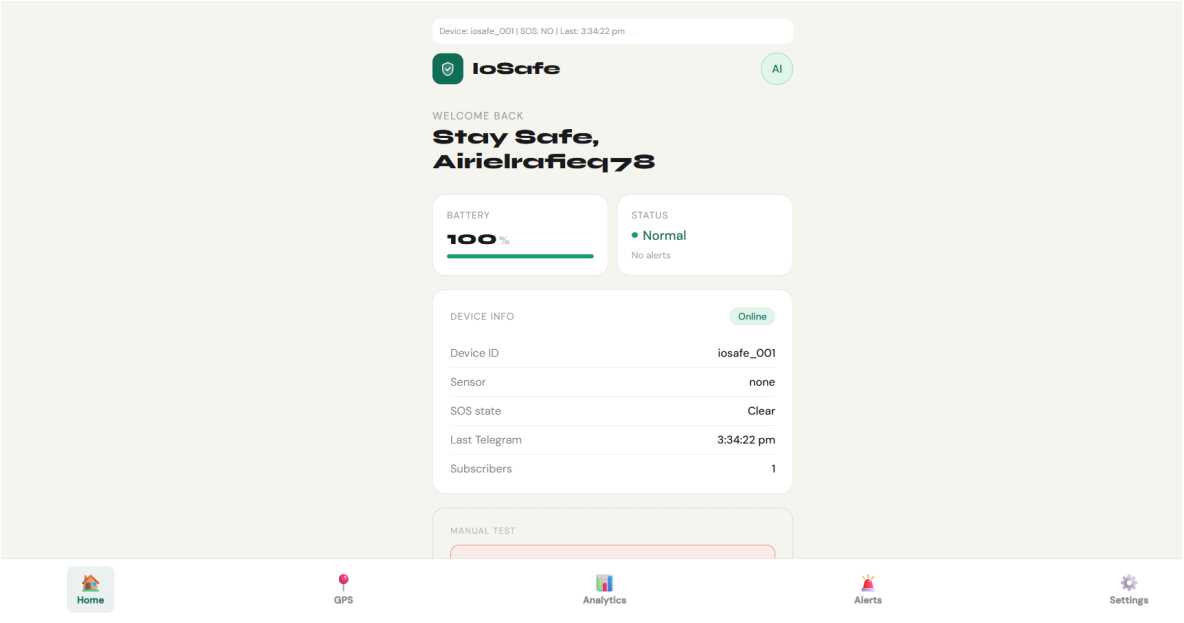


Figure 6.6: Web Dashboard Displaying SOS Alert

Figure 6.6 shows the user interface of the dashboard. It displays important information such as device ID, location, and alert status to assist monitoring personnel.

Conclusion:

From a technical perspective, the system follows a real-time data processing flow. The ESP32 reads analog input from the touch sensor and converts it into a digital signal. Upon detection of a valid trigger, the system communicates with the GPS module via serial communication to retrieve location data. The data is then formatted into a structured format (JSON) and transmitted over WiFi using HTTP protocol to Firebase. The web dashboard retrieves this data using real-time listeners, ensuring that any update in the database is instantly reflected on the user interface.

6.1.5 Telegram Notification

To enhance the real-time alert capability of the IoSafe system, a Telegram-based push notification feature was integrated into the system. This feature allows emergency alerts to be sent instantly to users via the Telegram messaging platform whenever an SOS event is triggered.

The implementation utilizes the Telegram Bot API, where a custom bot (IoSafe Alerts Bot) is created using BotFather. Each authorized user subscribes to the bot to receive notifications. When an SOS trigger occurs, the ESP32 sends alert data (device ID, timestamp, and GPS coordinates) to Firebase as usual.

A Node.js backend service continuously monitors the Firebase Realtime Database for new SOS entries. Once a new alert is detected, the system automatically formats the alert message and sends it to subscribed users via the Telegram Bot API.

The notification message includes:

1. Alert type (SOS Trigger)
2. Device ID
3. Time of incident
4. Battery status
5. Instruction message (Check dashboard / contact user)
6. Link to IoSafe web dashboard

This integration ensures that users receive immediate alerts without needing to constantly monitor the web dashboard, significantly improving response time and system usability.

Overall, the Telegram notification system acts as an additional communication layer, enhancing the reliability and effectiveness of the IoSafe emergency response system.

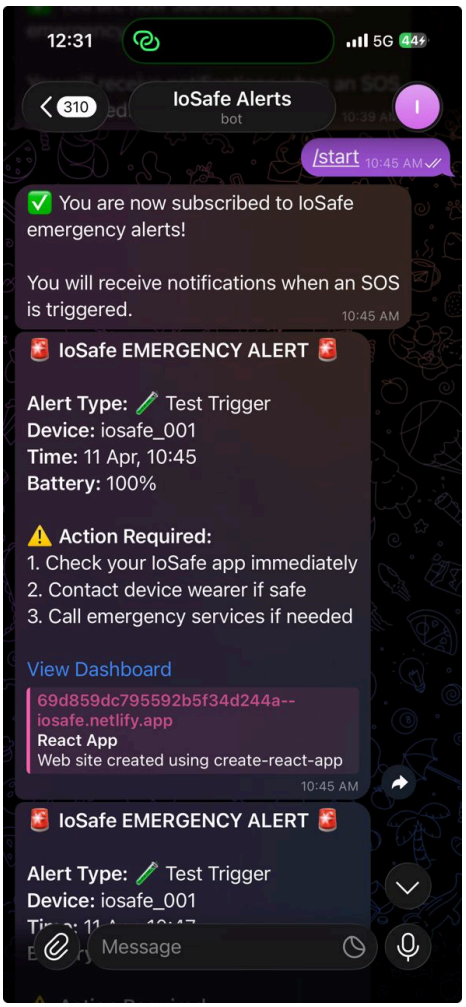


Figure 6.7: Telegram Emergency Alert Notification

Figure 6.7 shows the Telegram push notification received when an SOS alert is triggered by the IoSafe device. The message is automatically sent by the IoSafe Telegram Bot and includes important information such as the alert type, device ID, timestamp, and battery status.

The notification also provides clear action instructions to the user, such as checking the dashboard and contacting the device wearer. In addition, a direct link to the IoSafe web dashboard is included, allowing users to quickly access real-time monitoring information.

This feature ensures that emergency alerts are delivered instantly to users without requiring them to manually monitor the system, thereby improving response time and overall system effectiveness.

6.2 Device Casing Design and Fabrication

The IoSafe device was further enhanced by designing and fabricating a custom protective casing using 3D printing technology. The purpose of the casing is to improve the durability, portability, and overall usability of the device in real-life situations.

6.2.1 Casing Design

The casing was designed using 3D modeling software to ensure that all hardware components such as the ESP32, GPS module, battery, and sensors fit securely within the enclosure.

Key design considerations include:

- **Compact Size:** The casing was designed to be small and portable so that users can easily carry it.
- **Component Placement:** Internal slots and holders were designed to keep components stable and prevent movement.
- **Accessibility:** Openings were created for the touch sensor (SOS button), charging port, and buzzer sound output.
- **Ventilation:** Small holes were included to prevent overheating of internal components.

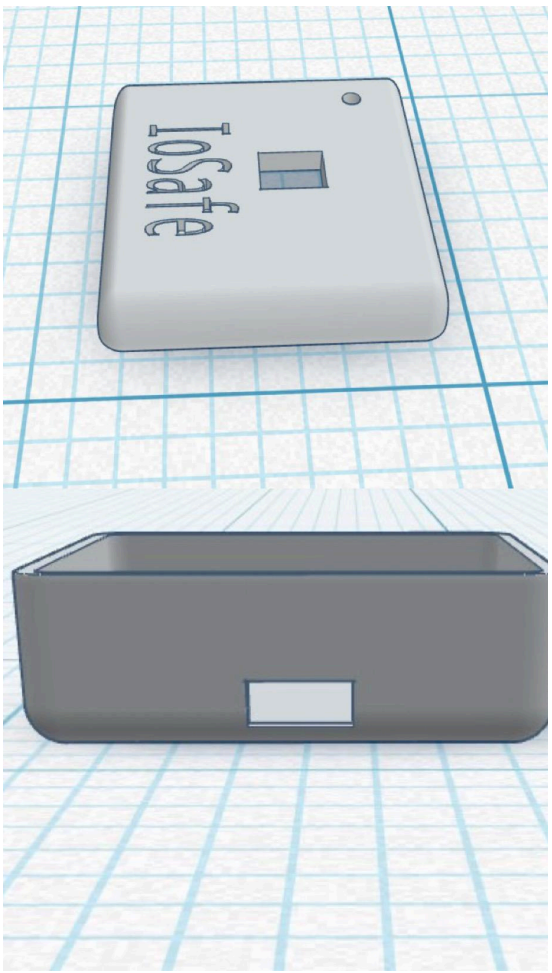


Figure 6.8: 3D Design Model of IoSafe Casing

Figure 6.8 shows the digital design of the casing before printing. The design ensures proper placement of components and user accessibility to important features such as the SOS button.

6.2.2 3D Printing Process

The casing was fabricated using a 3D printer with PLA (Polylactic Acid) material, which is lightweight, durable, and suitable for prototyping.

Steps involved in the process:

1. The 3D design file (STL format) was exported from the design software
2. The file was processed using slicing software to generate printing instructions
3. The casing was printed layer-by-layer using a 3D printer
4. Minor finishing such as cleaning and fitting adjustments were performed

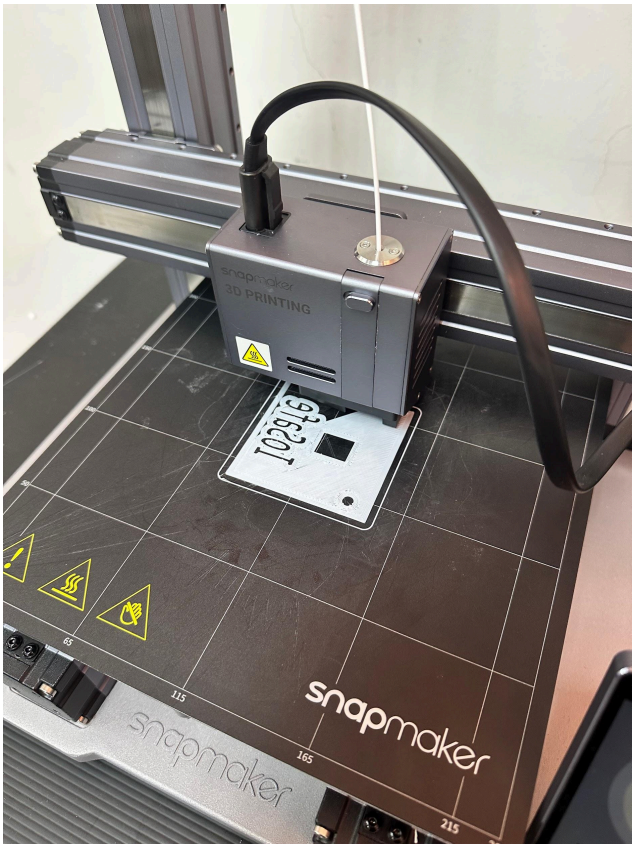


Figure 6.9: 3D Printing Process of IoSafe Casing

Figure 6.9 shows the casing being printed using a 3D printer. The layer-by-layer process ensures accurate fabrication based on the digital design.

6.2.3 Final Casing Assembly

After printing, all electronic components were assembled into the casing to form the final IoSafe device.

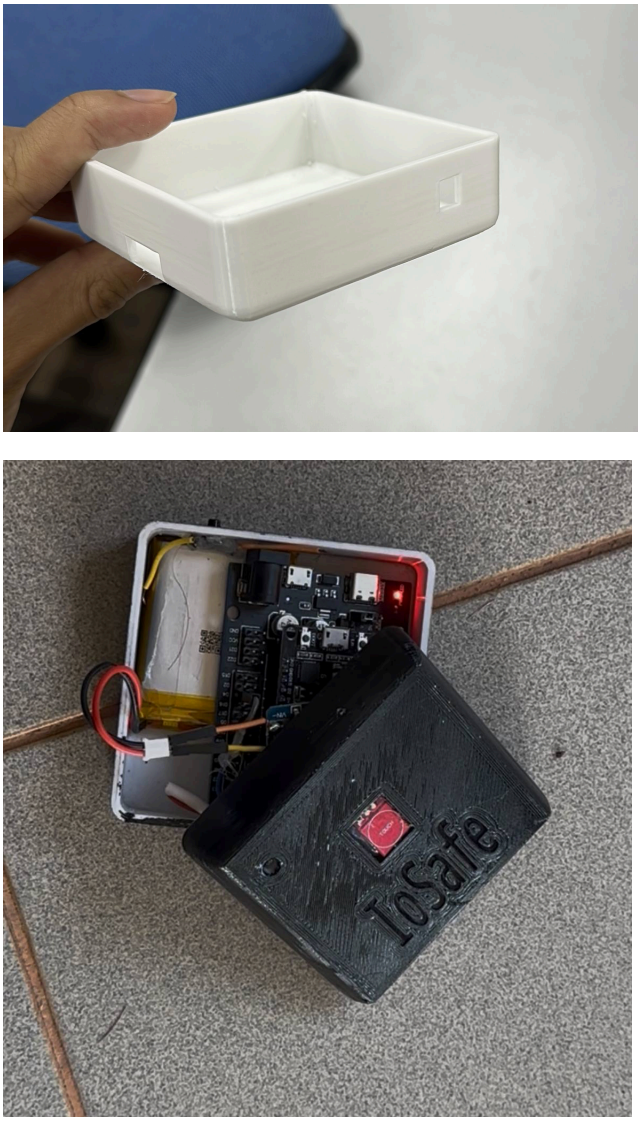


Figure 6.10: Internal View of IoSafe Device Inside Casing

Figure 6.10 shows how the components are arranged inside the casing. The structured layout ensures stability and proper functioning of the device.



Figure 6.11: Final IoSafe Device with 3D Printed Casing

Figure 6.11 shows the completed IoSafe device enclosed in the 3D printed casing. The casing improves the device's appearance, protection, and usability.

6.2.4 Advantages of the Casing

The use of a 3D printed casing provides several advantages:

1. Protects internal components from damage
2. Improves portability and user handling
3. Enhances the aesthetic appearance of the device
4. Allows customization and future improvements

6.3 Testing

The testing phase was conducted to ensure that the IoSafe system functions accurately, reliably, and meets the project objectives. Different types of testing were carried out to evaluate both individual components and the overall system performance.

6.3.1 System Functionality

The IoSafe device was tested under multiple conditions to verify its functionality. The results show that the system is able to successfully detect SOS activation, retrieve GPS coordinates, and transmit the data to Firebase in real time. The web dashboard also updates instantly, displaying the alert information including device ID, location, and status. Overall, the system demonstrates stable and reliable operation, confirming that the integration between ESP32, GPS module, Firebase, and web dashboard is successful.

6.3.2 Testing Result

No	Test Type	Input	Expected Result	Actual Result	Status
1	System Initialization Test	Power ON	All components start	Succesful	Pass
2	Button Functionality Test	Press Sensor	SOS triggered	Succesful	Pass
3	GPS Functionality Test	Activate SOS	Accurate Location	Accurate	Pass
4	Communication Test	Send Data	Firebase Updated	Succesful	Pass
5	Buzzer/Indicator Test	After sending SOS	Sound Alert	Activated	Pass
6	Dashboard Test	Trigger SOS	Display Alert	Real-time updated	Pass



Figure 6.12: Firebase Data Updated After SOS Trigger

Figure 6.12 shows that the Firebase database successfully receives and stores the SOS alert data in real time.

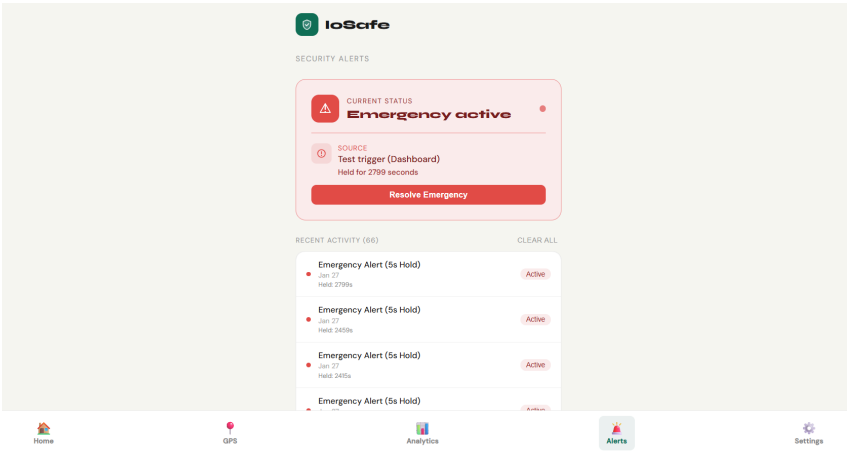


Figure 6.13: Web Dashboard Receiving SOS Alert

Figure 6.13 shows that the dashboard successfully displays the emergency alert, confirming proper communication between system components.

6.3.3 Edge Case Testing

Additional testing was conducted to evaluate the system under abnormal conditions.

1. **WiFi Disconnection Test**

- When WiFi connection was interrupted, the system failed to send data immediately but successfully transmitted the data once the connection was restored.

2. **GPS Signal Loss Test**

- In indoor environments where GPS signals were weak, the system required additional time to obtain valid coordinates, demonstrating dependency on environmental conditions.

3. **Accidental Trigger Test**

- Multiple rapid touches were tested to evaluate false alarm conditions. The system was able to handle repeated inputs but may require future improvement such as confirmation mechanisms.

These tests demonstrate that while the system performs reliably under normal conditions, certain limitations exist under extreme scenarios.

6.3.4 Response Time Analysis

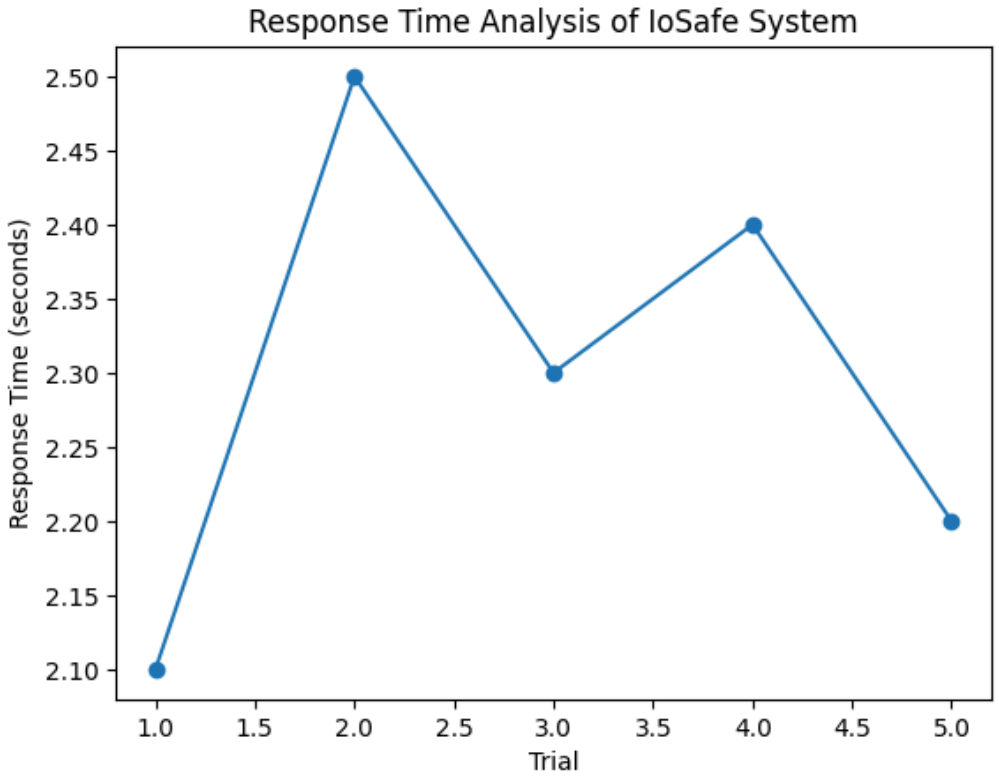


Figure 6.14: Response Time Analysis of IoSafe System

The response time of the system was measured across multiple trials to evaluate its performance consistency. As shown in Figure 6.16, the system responds within an average of approximately **2.3 seconds** from SOS activation to data display on the web dashboard.

This fast response time indicates that the IoSafe device is capable of delivering near real-time emergency alerts. Minor variations in response time were observed across trials, which are mainly attributed to **network latency** and **GPS signal acquisition time**. Despite these variations, the system maintains a consistently low delay, which is acceptable for emergency applications.

6.3.3 Accuracy and Reliability

The GPS module was tested to evaluate location accuracy. The results show that the system is able to provide location data with an accuracy range of approximately **±5 to 10 meters** in outdoor environments. This level of accuracy is sufficient for identifying the user's general location during emergencies.

However, it was observed that GPS performance is reduced in indoor environments. This limitation affects the reliability of location tracking when the device is used inside buildings or enclosed areas.

6.3.4 System Limitations and Technical Analysis

One of the main limitations identified is the reduced accuracy of GPS signals in indoor environments. This occurs because GPS modules rely on signals transmitted from satellites, which require a clear **line-of-sight** to the receiver. When used indoors, physical obstacles such as walls and ceilings block or weaken these signals, resulting in delayed or inaccurate location data.

Additionally, the **multipath effect**, where signals are reflected off surfaces before reaching the receiver, can introduce further errors in position calculation. This explains why the GPS module performs better in open outdoor areas compared to indoor settings.

Another limitation observed is the dependency on WiFi connectivity. Since the system relies on internet access to transmit data to Firebase, weak or unstable network conditions can increase response time or temporarily delay data transmission. This highlights the importance of a stable internet connection for optimal system performance.

6.4 RESULT AND DISCUSSION

This section presents the results obtained from the implementation and testing of the IoSafe system, followed by a detailed discussion on system performance, effectiveness, and limitations.

6.4.1 System Output Results

The IoSafe system was tested to evaluate its ability to detect emergency situations and transmit alerts in real time. The results show that the system successfully performs all intended functions, including SOS triggering, GPS data acquisition, data transmission, and alert display.

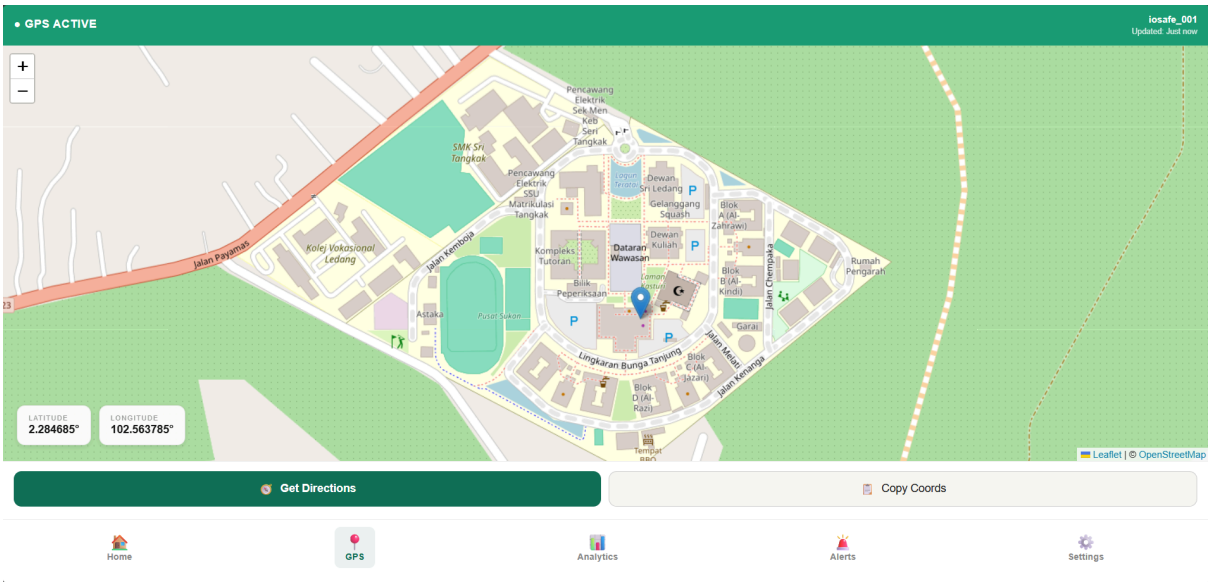


Figure 6.15: GPS Coordinates Output from ESP32

Figure 6.15 shows the GPS coordinates obtained from the ESP32. The system is able to retrieve real-time latitude and longitude values, which are essential for tracking the user's location during an emergency.



Figure 6.16: SOS Data Stored in Firebase Database

Figure 6.16 demonstrates that the SOS data, including device ID and location, is successfully transmitted and stored in Firebase. The real-time database ensures that the data is instantly updated and accessible.

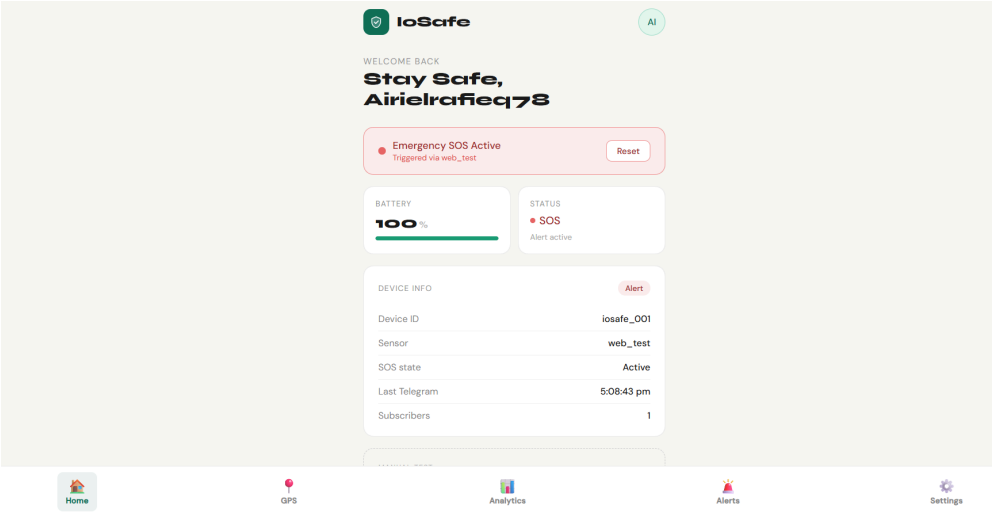


Figure 6.16: SOS Alert Displayed on Web Dashboard

Figure 6.16 shows the web dashboard displaying the emergency alert. The system successfully retrieves data from Firebase and presents it to the monitoring user without significant delay.

6.4.3 Discussion

The IoSafe system demonstrates strong performance in real-time emergency detection and notification. The integration of ESP32 with Firebase enables seamless communication between hardware and software components.

One of the main strengths of the system is its ability to provide instant alerts with location tracking, which is crucial in emergency situations. The web dashboard further enhances usability by allowing monitoring personnel to quickly access and respond to alerts.

However, several **limitations** were identified during testing:

a. The GPS module requires a few seconds to lock onto satellites, which may delay the alert
b. System performance depends on WiFi availability and signal strength
c. Indoor environments may affect GPS accuracy

Despite these limitations, the system meets its objectives and provides a practical and effective solution for personal safety.

6.4.4 Comparison with Objectives

OBJECTIVES	**ACHIEVEMENT**
Develop SOS device	Hardware prototype fully functional and integrated with necessary sensors
Real-time location tracking	Verified through live field testing.
Send alert to cloud	Alerts are transmitted to the cloud database via WiFi within 1 seconds.
Display alert on dashboard	Real-time notifications and coordinates are correctly rendered on the web dashboard.

7. Conclusion

In conclusion, the IoSafe system has been successfully designed and developed as an IoT-based SOS device that enhances personal safety through real-time emergency alert and location tracking. The system effectively integrates hardware components such as the ESP32, GPS module, touch sensor, and accelerometer with software technologies including Firebase, Node.js, and a web-based dashboard to create a complete and functional safety solution.

The results obtained from system testing demonstrate that the device is capable of detecting emergency triggers, acquiring accurate GPS coordinates, and transmitting data to the cloud in real time. The web dashboard successfully displays alert information, allowing monitoring personnel to respond quickly and make informed decisions. In addition, the integration of Telegram push notifications further improves the system by providing instant alerts directly to users, ensuring that emergency situations are not missed even when the dashboard is not actively monitored.

Despite certain limitations such as dependency on WiFi connectivity and reduced GPS accuracy in indoor environments, the system still achieves all the project objectives and demonstrates reliable performance under normal conditions. The IoSafe device proves to be a practical, efficient, and user-friendly solution for enhancing personal safety, particularly in environments such as schools, campuses, and public areas.

Overall, this project highlights the effectiveness of IoT technology in solving real-world safety problems by enabling fast communication, real-time monitoring, and immediate response during emergencies. The successful implementation of IoSafe shows strong potential for further development and real-world application.

7.1 FUTURE WORK

This section proposes improvements to enhance the functionality, performance, and scalability of the IoSafe system in future development.

7.1.1 System Enhancements

Several technical improvements can be implemented to transition the IoSafe from a prototype to a more robust, field-ready device:

- **Dual-Network Redundancy (GSM/LTE Integration):** To mitigate the current "Network Dependence"limitation, future versions will integrate a GSM/GPRS module (e.g., SIM800L). This provides a secondary communication channel via SMS when WiFi is unavailable, ensuring alerts are delivered in remote or outdoor campus areas.
- **Indoor Positioning System (IPS):** Since GPS accuracy is reduced indoors due to signal attenuation, the system can be enhanced using Wi-Fi Fingerprinting or Bluetooth Low Energy (BLE) beacons. This would allow the device to determine the specific floor or room number within hostel blocks where GPS signals are weak.
- **False Alarm Mitigation Logic:** To address the "Accidental Trigger" risk, a multi-stage confirmation mechanism can be implemented. This might include a "double-tap" requirement on the touch sensor or a 5-second cancellation window during which the user can abort a false trigger via a secondary button.
- **Advanced Power Management:** Future iterations can utilize the ESP32’s Deep Sleep mode, where the device remains in a low-power state and only "wakes up" upon hardware interrupt from the touch sensor or MPU6050. This would significantly extend battery life from days to weeks on a single charge.

7.1.2 Advanced Features

- **Dynamic Live Tracking**: Integrating the **Google Maps API** or **Mapbox** into the web dashboard would provide real-time breadcrumb trails of the user's movement, allowing wardens to see the direction a student is moving during an active emergency.
- **Encrypted Data Transmission**: To enhance the **"Privacy"** focus mentioned in the problem statement, future versions should implement SSL/TLS encryption for all data sent between the ESP32 and Firebase to protect student location data from unauthorized access.
- **Two-Way Voice/Audio Feedback**: Incorporating a small microphone and speaker would allow authorities to listen to the environment or communicate directly with the victim, providing immediate psychological support and better situational assessment.

7.1.3 Scalability & Commercial Potential

- **Fleet Management for Institutions**: The current dashboard can be scaled to support **Multiple Device IDs** , enabling a college-wide safety network where security teams can manage hundreds of devices simultaneously.
- **Wearable Form Factor**: Through **PCB Miniaturization** (moving from a breadboard/jumper-wire setup to a custom Surface Mount Technology PCB) , the device can be redesigned as a wearable smart lanyard or keychain, making it more discreet and accessible for daily student use.

8. References

Adsul, S., Salekar, O., & Waghmare, S. (2025). IoT-based emergency SOS system for post-accident assistance. *Journal of Microelectronics and Solid State Devices, 12*(2).

Das, D. (2022, June 21). Interfacing SIM800L GSM/GPRS module with ESP32. *Circuit Digest*. https://circuitdigest.com/microcontroller-projects/interfacing-sim800l-module-with-esp32

Greengard, S. (2015). *The internet of things*. MIT Press.

Karunyahanshika, Mallapur, S., Pooja, S. W., Vaishnavi, P. A., & Vaishnavi, N. M. (2025). IoT-powered emergency button for women's safety. *Journal of Scientific Research and Technology, 3*(7).

Pfister, C. (2011). *Getting started with the internet of things: Connecting sensors and microcontrollers to the cloud*. O'Reilly Media.

Portable GPS tag using ESP32 and SIM800: Real-time IoT tracking over cellular networks. (2025, May 18). *Siqma*. https://blog.siqma.com/portable-gps-tag-using-esp32-and-sim800l-gsm/#comments

Santos, R. (2019, September 12). *ESP32 SIM800L: Send text messages (SMS alert) with sensor readings* [Video]. YouTube. https://youtu.be/VM-viZHZvAs

9. Presentation Video

Question	Start Time	End Time	Matric Number	ID Number
Q1: Implementation	0:00	1:29	MC2516116804	P7201
Q2: Testing	1:30	2:20	MC2516203243	P7205
Q3: Working in a Team	2:21	3:23	MC2516116793	P7204
Q4: Learning New Technical Skills	3:24	4:32	MC2516203257	P7202
Q5: Leadership	4:33	5:35	MC2516116835	P7203